ECE4301.01
Date: 11/28/2025

ECE 4301.01 Midterm #2 Report

Team Members:
Caleb Jala-Guinto
Isabel Warth
Manuel Alvarado

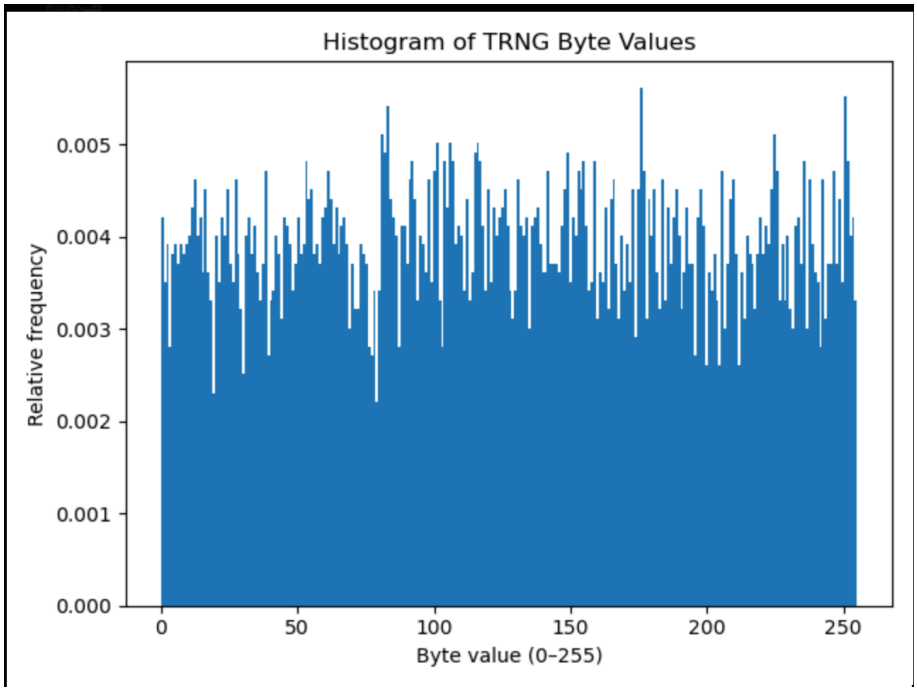
True Random Number Generator (TRNG):

```
calebjalapeno@raspberrypi:~/ECE4301_Fall2025/Group H/Midterm2 $ sudo python3 trn
g_demo.py
Reading 10 random 32-bit integers from /dev/hwrng:
0: 1725045020
1: 4106605525
2: 3584057499
3: 2700597016
4: 3263656197
5: 260255647
6: 2467444522
7: 568612860
8: 343330063
9: 2315171246
```

Monobit Output:

```
calebjalapeno@raspberrypi:~/ECE4301_Fall2025/Group H/Midterm2 $ sudo python3 trn
g_stats.py
Total bits: 32768
Zeros: 16684, Ones: 16084
P(0) ≈ 0.5092, P(1) ≈ 0.4908
```

Histogram of TRNG:



True Random Number Generators and Raspberry Pi Hardware TRNG

True Random Number Generators (TRNGs) rely on nondeterministic physical processes to produce unpredictable bitstreams. Several major TRNG designs are used in modern hardware. One common approach uses **thermal noise** (Johnson–Nyquist noise) from resistors or transistors. This naturally occurring electronic noise is amplified and digitized, then typically passed through a conditioning step to reduce bias. A similar idea appears in **avalanche or Zener breakdown TRNGs**, where a reverse-biased diode enters breakdown and produces random avalanche events that can be converted into random bits. These types are compact and widely used inside secure microcontrollers.

Other TRNGs are fully digital. **Ring-oscillator-based TRNGs** sample the timing jitter between multiple oscillators running at slightly different frequencies. Jitter arises from thermal fluctuations and voltage noise, making the edge timing unpredictable. Another digital approach uses **metastability**, where a latch or flip-flop is intentionally pushed into an unstable state. Tiny physical disturbances force it to resolve to either 0 or 1, producing a random outcome. At the high end, **photonic/quantum TRNGs** use fundamentally quantum-random effects—such as photon arrival times or beam-splitter outcomes—to generate extremely high-quality entropy.

Although the physical mechanisms differ, all TRNGs follow the same basic structure:

1. **Entropy Source** – a nondeterministic physical process (noise, jitter, avalanche events, photons)

2. **Sampling** – circuitry that converts the analog randomness into digital bits

3. **Conditioning** – post-processing such as hashing, XOR mixing, or whitening to remove bias

4. **Health Tests** – checks that ensure the entropy source is still functioning correctly

5. **Output Interface** – the processed bits are exposed to software, often through a device like `/dev/hwrng`

The Raspberry Pi's TRNG fits into this framework. Its Broadcom SoC uses a combination of oscillator jitter and thermal-noise circuits as the entropy source, followed by internal conditioning before making data available through **/dev/hwrng**. In our implementation, we read hardware-generated random numbers using Python (`trng_demo.py`) and then ran simple statistical checks with `trng_stats.py`. The monobit test showed an almost equal number of zeros and ones, and the histogram of byte values was roughly uniform. These results are consistent with a properly functioning TRNG and match the expected behavior of the Raspberry Pi's hardware random generator.